

```
letter.h2o <- as.h2o(letterimage)
```

Now it is time to set the dependent and independent variables from the letter data frame 'letterimage'. The first column containing the English letters is the dependent variable and the rest of the columns are the independent variables.

```
#dependent variable (Letter)
>y.dep<- 1
#independent variables (dropping ID variables)
>x.indep<- c(2:ncol(letterimage))
```

To simulate the deep learning model, the H2O data frame *letter.h2o* is divided into training and test data sets. First, 16,000 rows are assigned to the training data set and the remaining 4000 records are considered as the test data set.

```
>train<- letter.h2o[1:16000,]
>test<- letter.h2o[16001:nrow(letter.h2o),]
```

Now we are ready to form the deep learning neural network model. The H2O function *h2o.deeplearning()* is used here to create a feed-forward multi-layer artificial neural network model on the training H2O data frame 'train'.

```
>dlearning.model<- h2o.deeplearning
(
  y = y.dep,
  x = x.indep,
  training_frame = train,
  epoch = 50,
  hidden = c(100,100),
  activation = "Rectifier",
  seed = 1122
)
```

This function forms a neural model with two hidden layers with [100,100] synaptic nodes. The activation function of the model is set to the rectifier function. The initialisation of weightage and bias vectors has been done with random number sets with the seed value 1122. The model also sets the maximum number of iterative epochs over the model as 50. The training data set, along with the response and predictor variables (x, y), tunes the model as a universal classifier to identify and classify letters into their respective categories. The performance of the model can be studied with the help of the *h2o.performance()* function. To give you an idea about its performance, this function is used here over the created model itself.

```
>h2o.performance(dlearning.model)
```

This model can now be used over the test data to verify the performance of this deep neural model. The final performance study is done by comparing the outcome of the model with

the actual test data. As comparison requires variables in the *R data.frame* format, both the predicted and the test data are converted into data frames.

```
>predict.dl2 <- as.data.frame(h2o.
predict(dlearning.model, test))
>test.df<- as.data.frame(test)
```

A thorough look into both the data frames is helpful to gauge the correctness of the classification task. Often the verification of the performance is done by the confusion matrix. This is done by factorising both the data frames with the help of the table function, as shown in Table 2.

```
>table(test.df[,1],predict.dl2[,1])
```

Table 2

	A	B	C	D	E	F	G	H
A	147	0	0	1	0	0	0	
B	0	124	0	2	0	0	0	
C	1	0	137	0	1	0	1	
D	0	2	0	165	0	0	0	
E	0	0	3	0	134	2	5	
F	0	0	0	0	0	144	0	
...	...	...	..	...	...	...	..	..

From the confusion matrix it is apparent that though there are few false positive and false negation classifications, the overall performance is quite high. A tabular matrix display is also helpful to study the performance. This can be done from the test data frame with the help of the following command sequences:

```
>predictions <- cbind(as.data.
frame(seq(1,nrow(test.df))),
test.df[,1],predict.dl2[,1])
>names(predictions) <- c("Sr
Nos","Actual","Predicted")
>result <- as.matrix(predictions)
```

Sr	Nos	Actual	Predicted
[1,]	1	U	U
[2,]	2	N	N
[3,]	3	V	V
[4,]	4	I	I
[5,]	5	N	N
[6,]	6	H	H
[7,]	7	E	E
[8,]	8	Y	Y
[9,]	9	G	G
....	...		...